



Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`;
ELSE `move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent needs to store an action for every possible percept sequence. When the agent works for a longer time, the number of percept sequences becomes very large.

For a complex game like Chess, there are a huge number of possible board states and game sequences. Therefore, it is impossible to store all of them in one lookup table. This problem is called combinatorial explosion.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

In my SimpleReflexAgent, the following if statements are the Condition-Action Rules:

```
if percept['food_here']:
    return 'Suck'
```

```
if percept['wall_ahead']:
    return 'TurnLeft'
```

```
return 'MoveForward'
```

The first rule means:

IF food is here, THEN suck the food.

The second rule means:

IF there is a wall ahead, THEN turn left.

If these conditions are false, the agent moves forward.

These are Condition-Action Rules because the agent checks the current condition

3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The Simple Reflex Agent only looks at the current percept. It does not remember the places it visited before.

Also, the environment is partially observable. The agent cannot see the full map or its exact global position. It only knows information such as wall_ahead and food_here.

Because of this, the agent may come back to the same place and get the same percept again. Then it will choose the same action again. This can make the agent repeat the same path many times and get stuck in a loop.

For example:

wall_ahead → turn left → move forward → wall_ahead → turn left again

4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

My ModelBasedAgent has an internal memory. It stores values such as:

```
self.estimated_pos  
self.estimated_direction  
self.visited_cells  
self.last_action
```

The Transition Model is mainly handled in the `update_state()` method. The agent checks its previous action and updates its estimated position or direction.

For example, if the last action was `MoveForward`, the agent updates its estimated position. If the last action was `TurnLeft` or `TurnRight`, it updates its estimated direction.

The agent also uses sensor information from the current percept, such as:

```
percept['wall_ahead']  
percept['food_here']
```

This helps the agent understand the current situation before selecting the next action.

The `visited_cells` set stores the places that the agent has already visited. Therefore, if the agent sees a wall and knows that the left side was already visited, it can select

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

